Cloud Computing

Exercise - 10 Total points: 10

OpenFaaS for Serverless Computing in Kubernetes

1. Installation of Helm, the package Manager in Kubernetes

Kubernetes is an open-source container orchestration platform where specific applications can be managed using multiple independent resources like pods, services, deployments, and replica sets. Each of these must be configured and described in a file called "manifest" which is in JSON format or more commonly as a YAML file.

Writing YAML files for single application can be managed but for large-scale productions, we need to have a source to search, use and share existing implementations, create and deploy these configuration files and templates easily.

Helm, is an open-source package manager in Kubernetes like how apt works in Ubuntu. It allows operators to package, configure, and deploy applications and services onto Kubernetes clusters easily.

Source: https://gcore.com/learning/using-helm-with-kubernetes

1. Installing Helm
   To download and install Helm:
   curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

2. To verify helm installation:
   helm version

Helm can be used to install and configure different software easily. It provides templates which allow to configure custom deployments with minimal changes and without changing the template directly. Helm **charts** are packaged Kubernetes applications with templates and default configurations:

- Chart.yaml - metadata like name, version, etc.
- values.yaml - default values for the templates which can be overridden during installation.
- templates/ - a directory which contains manifest files

Charts are important as they allow us to reuse configurations, deploy complex applications and customize our own deployments. Helm repositories are where the helm charts are stored and shared for distribution. By installing Helm, it provides a CLI to search for different packages from the repositories.

The stable repository was the default Helm chart repository, but it's now deprecated and no longer maintained by the Helm project. Bitnami provides a public Helm chart repository containing a curated collection of charts for popular software applications.

3. Since Helm doesn't directly have connection to any of the repositories, we add "bitnami":

   helm repo add bitnami https://charts.bitnami.com/bitnami

4. To update the helm repository: helm repo update

5. To view the list of different chart repositories which are present: helm repo list

6. To search for available charts: helm search repo bitnami

As an example, let us look at the nginx chart first using bitnami repository via Helm.

7. To specifically search and install the nginx chart:

helm search repo bitnami/nginx

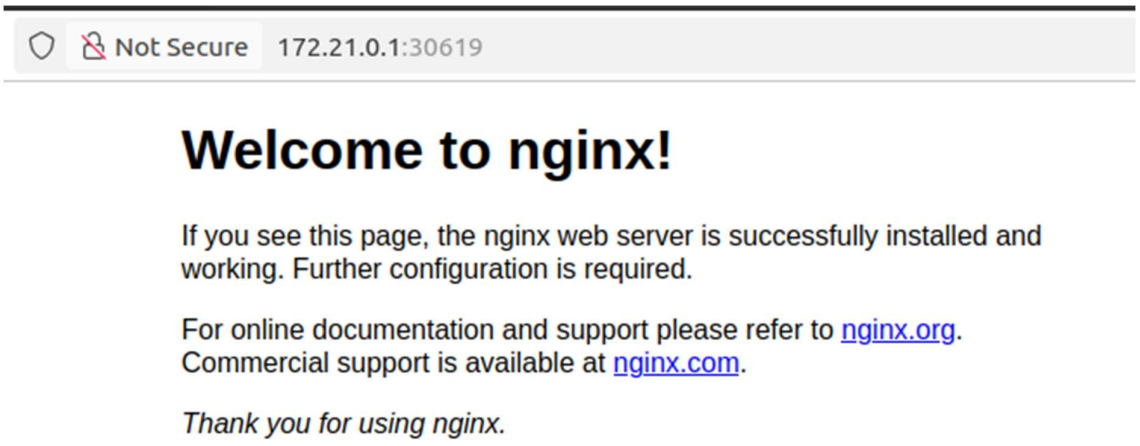
helm install my-nginx bitnami/nginx

8. To verify whether the pod has been created: kubectl get pods (it creates the pod in the default namespace)

9. To check the status of LoadBalancer IP allocation: kubectl get svc --namespace default -w my-nginx

```
osboxes@osboxes:~/satish/CC/kube/ex10openfaas$ kubectl get svc --namespace defau
lt -w my-nginx
NAME       TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)
    AGE
my-nginx   LoadBalancer   10.43.188.64   <pending>     80:30619/TCP,443:31287/TC
P   73s
^Cosboxes@osboxes:~/satish/CC/kube/ex10openfaas$ kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
my-nginx-565dfbfd98-fnvlh   1/1     Running   0          101s
```

**In this case, the External IP allocation is stuck at Pending because we are using the local k3s setup instead of using a cloud-based load balancer. However, the functionality is not affected as NodePort is configured, therefore, the Nginx service is exposed externally (in this case, HTTP port 80 is mapped to 30619). Therefore, use http://<node_ip>:<nodePort> to access the nginx UI on your system.



Customizing the Helm Chart for Nginx:

The configuration can be modified without changing the templates directly. To do this, the values can be overridden inline via cmd:

helm upgrade my-nginx bitnami/nginx --set service.type=NodePort --set replicaCount=2

This will explicitly set the service to NodePort type and create 2 replicas of the nginx pods. It upgrades the existing Helm release my-nginx by changing chart values (in this case, service type and replica count)

Scaling the Deployment:

kubectl scale deployment my-nginx --replicas=3

This will scale the number of pods in the deployment manually using kubectl.

To view the details of what Helm installed: helm get all my-nginx (This shows rendered manifests, values, hooks, etc.)

To list all the Kubernetes resources which have been created by the chart:

kubectl get all -l app.kubernetes.io/instance=my-nginx

At the end, helm delete my-nginx to delete the pods.

TODO (Exercise)**:**

a) While doing this part of the tutorial, show all the pods that are created and show their logs while they are handling the HTTP requests. (2 points)
NB! Take screenshots of the CLI output while executing the commands.

2. OpenFaaS deployment

Serverless functions: A serverless function is a code that runs on demand, without having to manage the underlying servers or infrastructure. These functions are event-driven(HTTP requests, timers, etc.), they are short-lived (meant for specific tasks) and scalable (any number of copies can be created). The platform which hosts these functions runs them only when needed.

OpenFaaS is an open-source platform designed to enable serverless functions on Kubernetes and other container orchestration systems. OpenFaaS handles tasks of **deploying**, **scaling**, **routing**, and **executing** them on demand. It simplifies the deployment and management of Functions-as-a-Service (FaaS) workloads, allowing developers to run serverless functions with ease while leveraging Kubernetes' scalability and robustness.

Helm simplifies the installation process by packaging OpenFaaS components into a chart.

1. Adding the OpenFaaS Helm Repository:

helm repo add openfaas https://openfaas.github.io/faas-netes/

This deploys all the necessary components like pods, services, secrets, etc.

2. Update the helm repository

helm repo update

3. Instead of using the default namespace in Kubernetes, let us create specific namespaces called openfaas and openfaas-fn in k3s environment:

kubectl create namespace openfaas (to hold the core components of OpenFaaS)

kubectl create namespace openfaas-fn (to hold the functions which we want to create)

4. To install OpenFaaS using Helm with basic authentication, timeout settings, pro version is set to false to ensure that the community edition is used.

```
helm install openfaas openfaas/openfaas \
--namespace openfaas \
--set generateBasicAuth=true \
--set faasnetes.readTimeout=30s \
--set faasnetes.writeTimeout=30s \
--set gateway.upstreamTimeout=30s \
--set gateway.readTimeout=30s \
--set gateway.writeTimeout=30s \
--set queueWorker.ackWait=30s \
--set openfaasPro=false
```

5. To verify that OpenFaaS has started:
kubectl -n openfaas get deployments -l "release=openfaas, app=openfaas"
This will give the main components of OpenFaaS: alertmanager, gateway, nats, prometheus and queue-worker which are present as pods in the openfaas namespace. In addition, the command kubectl get pods -o wide -n openfaas can also be used to view the pods.

- **Alertmanager** – Manages alerts from Prometheus and routes them to notification channels like email or Slack.
- **Gateway** – The entry point for OpenFaaS, handling function requests, authentication, and scaling.
- **NATS** – A lightweight messaging system for handling asynchronous function invocations in OpenFaaS.
- **Prometheus** – Monitors OpenFaaS components, collecting metrics for function execution and system health.
- **Queue-worker** – Processes background/asynchronous function requests received via NATS.

For each of the core component pods in OpenFaaS, like queue-worker for example, there needs to be a single instance of the pod running successfully. To verify this:

kubectl get deployment queue-worker -n openfaas -o yaml

kubectl get deployment queue-worker -n openfaas -o yaml | grep "replicas:"

Both the desired (spec) and the current (status) replicas should show 1, confirming that exactly one instance is meant to run and is running.

TODO (Exercise)**:**

b) Repeat the task for all the other core components and show their replication levels (2 points)
NB! Take screenshots of the CLI output while executing the commands.

6. In OpenFaaS, to retrieve the admin or Gateway password, run the command:

```
echo $(kubectl -n openfaas get secret basic-auth -o jsonpath="{.data.basic-auth-password}" | base64 --decode)
```

7. The OpenFaaS gateway needs to be accessible to interact with functions.
To expose the OpenFaaS gateway:

```
kubectl port-forward -n openfaas svc/gateway 8080:8080 &
```

The OpenFaaS UI can be accessed with http://localhost:8080 with login credentials as username: admin and password: (retrieved in earlier in step 6)

OR to use command-line based approach:

Installing faas-cli:

```
curl -sSL https://cli.openfaas.com | sudo sh
```

Verify installation of faas-cli tool:

```
faas-cli version
```

8. To login to OpenFaaS via faas-cli:

```
echo -n "<password_from_step6>" | faas-cli login --gateway http://127.0.0.1:8080 -u admin --password-stdin
```

3. Testing a Function using OpenFaaS

To test the functionality of OpenFaaS, let us use a serverless function called figlet, which takes a string as input (like "hello") and returns an ASCII art version of that string in the terminal. When figlet is deployed using faas-cli, it is scheduled as a containerized function inside the OpenFaaS namespace in k3s. When it is invoked, OpenFaaS routes the request to the running function pod.

1. To deploy the figlet function:

```
faas-cli store deploy figlet
```

2. The figlet pod gets created in the openfaas-fn namespace:

```
kubectl get pods -o wide -n openfaas-fn
```

3. To obtain the list of all the deployed functions:

```
faas-cli list
```

4. To invoke the function:

```
echo "Hello from Device" | faas-cli invoke figlet
```

5. To view the logs of the pod:

```
kubectl logs -n openfaas-fn <pod_name>
```

What happens here: ‘figlet’ is just a small containerized script/function which, OpenFaaS wraps in a pod and exposes it via HTTP.

To delete the figlet pod:

faas-cli delete figlet

TODO (Exercise)**:**

c) Deploy new function "Nodeinfo" from the store using the OpenFaaS UI. Show all the screenshots. (1 point)
d) Now invoke the function via curl command using: curl http://127.0.0.1:8080/function/nodeinfo
e) Expose the OpenFaaS Prometheus service on port 9090. (1 point)
f) Then, go to http://localhost:9090 (to view the Prometheus UI)

- Prometheus scrapes metrics from targets (such as OpenFaaS components) at regular intervals by querying HTTP endpoints that expose the data in a specific format.
  It's useful for monitoring the performance, health, and behavior of systems, enabling the visualization of trends and set alerts based on metric data over time.
  g) With 4-5 invocations of the nodeinfo function, Prometheus will record each invocation as an event, and any related metrics, like **gateway_function_invocation_total**, will show increasing values over time, which can be visualized in graphs to track function usage or behaviour (e.g., how frequently it's being called). If you graph this data, you'll see the number of invocations grow with each request, allowing you to analyse usage patterns. (1 point)

NB! Take a screenshot of the CLI output.
NB! Take the screenshots of the Prometheus UI.

4. Custom serverless function template on OpenFaaS via k3s

The goal is to use the YOLOv5 model (You Only Look Once - a popular object detection model for Deep Learning tasks to identify objects from a set of input images via Serverless function calls.

1. To view the available official templates (base environment for functions) in the store:
faas-cli template store ls -v
Each of these templates includes a Dockerfile, handler logic and some metadata.

2. To download a template (python3-http in this case):
faas-cli template store pull python3-http
This downloads the python3-http template into a local template/ directory and sets up a function that will handle HTTP requests, along with stdin/stdout. The template folder has many versions but let us use the python-http template.

3. In the python3-http folder, to create a new function called yolov5-function:
faas-cli new yolov5-function --lang python3-http --prefix=snsrirama
Here, --lang python3-http tells OpenFaaS to use the downloaded HTTP Python template and --prefix <docker_hub username> uses your Docker Hub username (this is so the image gets tagged as <docker_hub username>/yolov5-function).

4. cd yolov5-function folder, here we will create a Dockerfile, code for handler.py, index.py, requirements.txt, stack.yaml as follows:

a. index.py:

What it does: It is a simple Flask app which defines the POST route as '/'. It also forwards the image byte stream to handle() function in handler.py

```
from flask import Flask, request, jsonify
from handler import handle

app = Flask(__name__)

@app.route("/", methods=["POST"])
def main():
    req_data = request.data
    response = handle(req_data)
    return response

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

b. Dockerfile:

What it does: It builds a custom OpenFaaS container for object detection. lightweight HTTP server from OpenFaaS. It installs the minimal dependencies for OpenCV, copies the python files (handler.py, index.py and requirements.txt)

```
# Use OpenFaaS watchdog for HTTP handling
ARG PYTHON_VERSION=3.10
FROM --platform=${TARGETPLATFORM:-linux/amd64} ghcr.io/openfaas/of-watchdog:0.10.7 AS watchdog

FROM --platform=${TARGETPLATFORM:-linux/amd64} python:${PYTHON_VERSION}-slim AS build

COPY --from=watchdog /fwatchdog /usr/bin/fwatchdog
RUN chmod +x /usr/bin/fwatchdog

RUN apt-get update && apt-get install -y \
    libgl1-mesa-glx \
    libglib2.0-0 \
    libsm6 \
    libxext6 \
    libxrender-dev \
    && apt-get clean && apt-get install -y mosquitto-clients && rm -rf /var/lib/apt/lists/*


WORKDIR /home/app/

COPY handler.py .
COPY index.py .
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt --extra-index-url https://download.pytorch.org/whl/cpu

# Download YOLO model at build time
RUN python -c "from ultralytics import YOLO; YOLO('yolov5s.pt')"

RUN addgroup --system app && adduser --system --ingroup app app
RUN chown -R app:app /home/app
RUN mkdir -p /home/app/received && chown -R app:app /home/app/received
USER app

FROM build AS ship
WORKDIR /home/app/

USER app
ENV fprocess="python3 index.py"
ENV cgi_headers="true"
ENV mode="http"
ENV upstream_url="http://127.0.0.1:5000"
```

```
HEALTHCHECK --interval=5s CMD [ -e /tmp/.lock ] || exit 1

CMD ["fwatchdog"]
```

c. handler.py:

What it does: This has the logic for the actual object detection. It converts the incoming byte stream (when the raw images are provided) to image type using cv2.imdecode. It runs YOLOv5 on the image, parses bounding boxes and classes and returns JSON response.

```
import cv2
import numpy as np
from ultralytics import YOLO
from flask import jsonify

model = YOLO("yolov5s.pt")

def handle(req_data):
  try:
    # Read image from request
    image = np.frombuffer(req_data, np.uint8)
    img = cv2.imdecode(image, cv2.IMREAD_COLOR)

    if img is None:
      return jsonify({"error": "Invalid image"}), 400

    # Perform object detection
    results = model(img)

    # Process results
    detections = []
    for result in results:
      for box in result.boxes.data:
        x1, y1, x2, y2, conf, cls = box.tolist()
        detections.append({
          "x1": int(x1),
          "y1": int(y1),
          "x2": int(x2),
          "y2": int(y2),
          "confidence": float(conf),
          "class": int(cls),
          "label": model.names[int(cls)]
        })

    return jsonify({"detections": detections})

  except Exception as e:
    return jsonify({"error": str(e)}), 500
```

d. stack.yaml file:

What it does: It has the OpenFaaS function definition for CLI tools.

```
version: 1.0
provider:
 name: openfaas
 gateway: http://127.0.0.1:8080
functions:
 yolov5-function:
  lang: dockerfile
  handler: .
  image: snsrirama/yolov5-function:latest
  read_only: false
```

e. requirements.txt:

What it does: List of python libraries required to be installed as part of the Dockerfile.

```
torch==2.6.0
torchvision==0.21.0
flask
ultralytics
opencv-python-headless
numpy
pillow
```

5. Building the docker image: faas-cli build -f stack.yaml

Verify that the docker image has been created using: docker images

6. Tag the docker image: docker tag <your_dockerhub_id>/yolov5-function:latest docker.io/<your_dockerhub_id>/yolov5-function:latest

7. Push the image to docker hub: docker push <your_dockerhub_id>/yolov5-function:latest

8. Now, to deploy the pod for this function in the openfaas-fn namespace:

faas-cli deploy -f stack.yaml

9. Verify the pod's creation and status as running via: kubectl get pods -o wide -n openfaas-fn

10. To check the logs of the pod: faas-cli logs yolov5-function

11. Create a folder called "images" anywhere (including the present directory) and collect a few images. In this task, let us consider images of people:

Use custom images by downloading them from the internet.

12. cd <path_to_images_folder>

Here, we will use the curl command to invoke the yolov5-function in a serverless manner.

Example of an image tested upon:

1.jpg has 4 people.



1.jpg

```
osboxes@osboxes:~/satish/CC/kube/ex10openfaas/template/python3-http/images$ ls
10.jpeg  1.jpg  2.jpg  3.jpg  4.jpg  5.jpeg  6.jpg  7.jpeg  8.jpg  9.jpg
osboxes@osboxes:~/satish/CC/kube/ex10openfaas/template/python3-http/images$ curl
 -X POST http://localhost:8080/function/yolov5-function --data-binary @1.jpg -H
"Content-Type: application/octet-stream"
{"detections":[{"class":0,"confidence":0.9003552198410034,"label":"person","x1":
598,"x2":1024,"y1":157,"y2":854},{"class":0,"confidence":0.8767670392990112,"lab
el":"person","x1":889,"x2":1299,"y1":99,"y2":857},{"class":0,"confidence":0.8303
784132003784,"label":"person","x1":0,"x2":494,"y1":36,"y2":857},{"class":0,"conf
idence":0.7444570660591125,"label":"person","x1":276,"x2":670,"y1":254,"y2":860}
]}
```

The YOLO model can detect objects from 80 different classes. The OpenFaaS function accepts a raw image as input via POST request, it uses the YOLOv5 model for object detection. The output of the function invocation is JSON-format text which gives us information about the type of objects detected (in this case, “person,” “tie,” etc. classes), along with the bounding box coordinates of the detections which were made.

At the end, to remove the function: faas-cli remove yolov5-function

TODO (Exercise)**:**

1. Update the above code to draw the bounding boxes in the image around the objects that are detected. (1 point)
2. Save the images to the running pod. (1 point)
3. Download the images automatically to the local folder using the kubectl commands. (1 point)
4. Submit the source code and the updated files.
5. NB! Show the commands and the screenshots.
6. NB! Display the resulting images.

Follow these guidelines while submitting the exercises

1. Put a pdf file with your answers and screenshots for the questions and name it as your studentid_QE10
2. The screenshots are meant to show that the work is done and by you. So, have enough proofs in your answer sheet.
3. **Note**: Do not blindly copy the commands. Try to understand them. The provided commands are to guide you with the initial process.
4. Do not copy code from internet. If you copy, please leave a comment with the URL on that code/answer sheet
5. Plagiarism will not be tolerated and dealt through the formal channels of the university